Corso di Ingegneria degli Algoritmi - Lezioni di C

Scheda di Riepilogo Integrato e Concetti Chiave

Docente: Dott.ssa F. Montesano — Università degli Studi di Roma "Tor Vergata"

---

1 Introduzione al C e Processo di Compilazione

Il C è un linguaggio procedurale imperativo ad alto livello con potenti capacità di manipolazione della memoria a basso livello. Sviluppato da Dennis Ritchie presso i Bell Labs (1972–1973) per riscrivere il kernel UNIX, fornisce un modello mentale diretto del funzionamento della RAM e dell'architettura hardware.

1.1 Processo di Compilazione Passo-Passo con GCC

Il codice sorgente (`.c`) viene trasformato in un eseguibile attraverso quattro fasi distinte:

1. **Preprocessore** (`gcc -E main.c -o main.i`): Espande le direttive (`#include`, `#define`), rimuove i commenti e gestisce la compilazione condizionale. Il file sorgente espanso cresce notevolmente di dimensione.
2. **Compilatore** (`gcc -S main.i -o main.s`): Traduce il C espanso in linguaggio Assembly specifico dell'architettura target (istruzioni `mov`, `push`, `call`).
3. **Assembler** (`gcc -c main.s -o main.o`): Converte l'Assembly in codice macchina rilocabile (file oggetto `.o`).
4. **Linker** (`gcc main.o -o main`): Risolve le dipendenze dei simboli esterni e unisce i file oggetto con le librerie di sistema per generare l'eseguibile finale.

1.2 Sintassi Fondamentale e Parole Chiave

Lo standard ANSI C89 definisce 32 parole chiave riservate (`if`, `int`, `return`, `struct`, ecc.). Le *keywords* sono **case-sensitive** (scritte esclusivamente in minuscolo) e non possono essere usate come identificatori di variabili o funzioni.

Obiettivo: Comprendere la natura del linguaggio C, il suo legame storico con UNIX e saper padroneggiare le quattro fasi di trasformazione del codice sorgente in eseguibile tramite il compilatore GCC.

2 Tipi di Dato, Costanti, Operatori e Comportamenti Hardware

2.1 Tipi Primitivi e Dimensione in Memoria

La dimensione esatta dei tipi di dato dipende dall'architettura del sistema (32/64 bit). L'operatore `sizeof` (valutato a tempo di compilazione) restituisce la dimensione in byte di un tipo o di una variabile (utilizzando lo specificatore di formato `%zu`).
Dallo standard C99, l'header `<stdint.h>` mette a disposizione tipi ad ampiezza garantita (`int8_t`, `int32_t`, `uint64_t`) e `intptr_t` (intero della dimensione esatta di un puntatore).

Tipo Base	Specificatore	Dimensione Minima	Note e Intervallo
`char`	`%c`	8 bit (1 Byte)	Carattere ASCII o piccolo intero ($-128 \dots 127$)[cite: 26]
`short int`	`%hd`	16 bit	Intero con intervallo ridotto[cite: 26]
`int`	`%d`	16 bit o 32 bit	Intero standard ($-2^{31} \dots 2^{31}-1$)[cite: 26]
`long`	`%ld`	32 bit	Intero long ($-2^{31} \dots 2^{31}-1$)
`long long int`	`%lld`	64 bit	Intero ad ampia capacità[cite: 26]
`float`	`%f`	32 bit	Reale a singola precisione (IEEE 754)[cite: 26]
`double`	`%lf`	64 bit	Reale a doppia precisione[cite: 26]

2.2 Costanti, Casting ed Errori Aritmetici

- **Immutabilità:** Una variabile `const` è a sola lettura e deve essere inizializzata contestualmente alla dichiarazione. A differenza di `#define` (sostituzione testuale del preprocessore), `const` è gestito dal compilatore, possiede un tipo e rispetta le regole di scope.
- **Divisione Intera e Casting:** L'operazione `1 / 2` produce `0` (troncamento decimale). Per ottenere il risultato floating-point corretto occorre eseguire un casting esplicito: `((double)1) / 2`.
- **Integer Overflow e Wrap-Around:** Nei tipi con segno (`signed`), l'overflow causa un *Undefined Behavior* (UB) che nel sistema binario in complemento a due provoca solitamente un *wrap-around* verso valori negativi. Negli `unsigned`, l'aritmetica è modulo 2^N ($0 - 1 =$ UINT_MAX).
- **Precisione Floating-Point (IEEE 754):** Alcune frazioni decimali (es. 0.1) non sono rappresentabili esattamente in binario ($0.1+0.2 \neq 0.3$). Non bisogna mai usare l'uguaglianza diretta `==` sui reali, ma verificare che la differenza assoluta sia inferiore a una tolleranza: `fabs(a - b) < FLT_EPSILON`.

2.3 Operatori Bitwise e Valutazione Cortocircuitata

- **Operatori Bitwise:** `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (shift sinistro, moltiplica per 2^n), `>>` (shift destro, divide per 2^n). Consentono manipolazioni dirette di registri hardware a costo di un singolo ciclo di clock.
- **Valutazione Cortocircuitata (*Short-Circuit*):** Negli operatori logici `&&` e `||`, se il primo operando determina il risultato della condizione (es. `false` in AND), il secondo operando non viene nemmeno valutato. Ciò previene crash a runtime (es. `if (ptr != NULL && ptr->val > 0)`).
- **Pre-Incremento vs Post-Incremento:** `++x` incrementa la variabile e ne restituisce il valore aggiornato; `x++` salva una copia temporanea del valore originale, incrementa la variabile e restituisce la copia.

Obiettivi: Saper identificare i tipi primitivi e la loro rappresentazione in memoria, gestire correttamente il casting, prevenire errori di overflow e comprendere l'aritmetica bitwise e la valutazione cortocircuitata.

3 Strutture di Controllo, Funzioni, Scope e Gestione Modulare

3.1 Strutture di Controllo e Salto

- **Selezione:** Costrutto `if-else`, operatore ternario `(cond) ? expr1 : expr2` e `switch` (funzionante solo su tipi interi/caratteri). Negli `switch`, l'assenza della clausola `break` causa il fenomeno del *fall-through*. Il `case default` non è mandatorio ma buona pratica.
- **Iterazione:** `for` (utilizzato per cicli determinati con contatore), `while` (entry-controlled, per cicli condizionali e di lunghezza non determinata a priori) e `do-while` (exit-controlled, con almeno una prima esecuzione garantita). Fondamentale aver capito che `for` e `while` sono intercambiabili.
- **Salto e Canali Standard di Output:** `break` e `continue` agiscono esclusivamente sul ciclo più interno in cui sono inseriti, ignorando i blocchi `if`.

I messaggi di errore sono solitamente inviati su `stderr` tramite `fprintf(stderr, ...)` e non su `stdout`.

3.2 Funzioni, Passaggio dei Parametri e Prototipi

Le funzioni garantiscono modularità e riuso. In C, una funzione deve essere **dichiarata (prototipo)** o definita prima della sua invocazione.

- **Passaggio per Valore:** In C **tutti i parametri sono passati per valore**. La funzione riceve una copia dei parametri formali. Per modificare variabili del chiamante occorre passare l'indirizzo di memoria.

3.3 Scope delle Variabili e Progetti Multi-File

Tipo di Scope	Ciclo di Vita / Memoria	Descrizione
Block Scope	Temporaneo nello Stack	Visibile solo nel blocco {} di dichiarazione.
File Scope	Persistente (Data/BSS)	Dichiarata fuori dalle funzioni, visibile nell'intero file.
Static Locale	Persistente (Data/BSS)	Visibilità locale alla funzione, ma valore mantenuto.
Static Globale	Persistente (Data/BSS)	Visibile solo all'interno del singolo file `.c` (incapsulamento).
Extern	Persistente (Data/BSS)	Dichiara la presenza di una variabile globale definita in un altro file.

Regola d'oro per i file Header (`.h`): Nei file `.h` le variabili globali devono essere **esclusivamente dichiarate** con `extern` e mai definite (`int x = 10;`), altrimenti il linker solleverà un errore di *multiple definition* in fase di compilazione modulare. Le *include guard* (`#ifndef MY_HEADER_H_`) prevengono la re-inclusione ciclica degli header.

Obiettivi: Padroneggiare i costrutti di controllo del flusso, la modularizzazione del codice tramite funzioni e prototipi e capire la differenza tra dichiarazione di una funzione e definizione di una funzione. Capire le regole di visibilità delle variabili e la differenza tra visibilità e persistenza di valore (`extern` vs `static`) e la gestione di progetti multi-file.

4 Array, Stringhe e Modello di Memoria

4.1 Array Monodimensionali e Multidimensionali

Un array è una struttura dati omogenea costituita da elementi contigui in memoria con indicizzazione 0-based.

- **Accesso Diretto $O(1)$ e Layout:** L'indirizzo fisicamente occupato dall'elemento i-esimo è derivato tramite calcolo aritmetico:

$$\text{Indirizzo}(A[i]) = \text{Indirizzo Base}(A) + i \times \text{sizeof(tipo)}$$

- **Layout Row-Major per Matrici 2D:** In memoria la matrice viene linearizzata riga per riga. L'accesso all'elemento (i, j) segue la formula:

$$\text{Indirizzo}(A[i][j]) = \text{Indirizzo Base}(A) + ((i \times N_{\text{colonne}}) + j) \times \text{sizeof(tipo)}$$

- **Sintassi VLA (C99):** Nel passaggio di matrici a funzioni, le dimensioni devono precedere la matrice nella firma: `void stampa(int r, int c, int mat[r][c])`.

4.2 Stringhe e Input Sicuro

Una stringa C è un array di caratteri terminato dal carattere nullo `'\0'` (ASCII 0). Un array che deve contenere K caratteri deve allocare almeno $K + 1$ byte.

- **Pericolo di `gets()` e `scanf("%s")`:** Non controllano la capacità del buffer, causando *Buffer Overflow* e corruzione dello stack.
- **Input Sicuro con `fgets()`:** La funzione `fgets(buffer, DIM, stdin)` legge al massimo `DIM - 1` caratteri salvaguardando il terminatore `'\0'`. Se l'input contiene il tasto Invio, `fgets` memorizza anche il carattere `'\n'`, che può essere rimosso con: `buffer[strcspn(buffer, "\n")] = '\0'`.
- **Libreria `<string.h>`:** `strlen(s)` calcola la lunghezza escluso `'\0'`; `strcpy(dest, src)` copia le stringhe; `strcmp(s1, s2)` effettua il confronto lessicografico in base ai codici ASCII (restituisce `0` se identiche).

Obiettivi: Comprendere come un array monodimensionale sia rappresentato fisicamente nella RAM come una sequenza contigua di celle, sfruttando l'indicizzazione a base zero (0-based) e l'accesso diretto in tempo costante $O(1)$ tramite calcolo aritmetico degli indirizzi. Acquisire padronanza nella gestione delle matrici multidimensionali (2D), comprendendo il concetto di linearizzazione in memoria secondo l'ordine Row-Major. Capire la natura delle stringhe nel linguaggio C: non un tipo di dato nativo, ma semplici array di caratteri (char) rigorosamente terminati dal carattere nullo speciale `'\0'`. Saper manipolare array e stringhe utilizzando tutti i concetti precedentemenenti spiegati (tipi di dato e controllo del flusso).

5 Puntatori, Aritmetica degli Indirizzi e Memoria Dinamica

5.1 Puntatori e Aritmetica dei Puntatori

Un puntatore è una variabile che contiene l'indirizzo di memoria di un'altra risorsa.

- **Operatori Fondamentali:** `&` (Address-of) estrae l'indirizzo; `*` (Dereferenziazione) accede al valore presente all'indirizzo contenuto.
- **Aritmetica degli Indirizzi:** Incrementare un puntatore di 1 (`p++`) fa avanzare l'indirizzo di un offset pari a $1 \times$ sizeof(*p) byte. La sottrazione tra due puntatori dello stesso tipo restituisce il numero di elementi interposti (tipo `ptrdiff_t`).
- **Equivalenza Vettori-Puntatori:** L'operazione di indicizzazione `arr[i]` è zucchero sintattico per la dereferenziazione dell'aritmetica dei puntatori:

$$\texttt{arr[i]} \iff *(\texttt{arr} + \texttt{i}) \iff \texttt{i[arr]}$$

- **Array Decay:** Quando un array viene passato a una funzione, decade implicitamente al puntatore al suo primo elemento (`Type*`) perdendo l'informazione sulla propria dimensione totale (fanno eccezione solo l'operatore `sizeof(arr)` e `&arr`).

5.2 Layout della RAM, Heap e Allocazione Dinamica

Questo NON va saputo spiegare nel dettaglio, essendo argomento più avanzato, ma va capito che esistono zone diverse della memoria di un programma in cui vengono salvate le variabili.

La memoria assegnata ad un processo C si articola in 5 segmenti principali:

1. **Text Segment:** Codice eseguibile (in sola lettura).
2. **Data Segment:** Variabili globali e statiche inizializzate.
3. **BSS Segment:** Variabili globali e statiche non inizializzate (azzerate all'avvio).
4. **Stack:** Gestito automaticamente dal compilatore per memorizzare le variabili locali e i frame delle funzioni. Cresce verso l'alto.
5. **Heap:** Memoria dinamica a dimensione variabile gestita manualmente dal programmatore a runtime. Cresce verso il basso.

Listing 1: Esempio di Gestione della Memoria Dinamica

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 10;

    // Allocazione nello Heap
    int *v = (int *) malloc(n * sizeof(int));

    // Controllo difensivo dell'allocazione (malloc restituisce NULL se la RAM e' piena)
    if (v == NULL) {
        fprintf(stderr, "Errore: Memoria Heap esaurita!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        v[i] = i * 10;
    }

    // Rilascio della memoria per evitare Memory Leak
    free(v);
    v = NULL; // Azzera il puntatore per prevenire Dangling Pointer
    return 0;
}
```

Funzione	**Comportamento e Note di Sicurezza**
`malloc(size)`	Alloca `size` byte nello Heap (memoria sporca non inizializzata).
`calloc(n, size)`	Alloca memoria per n elementi ed azzera la memoria.
`realloc(ptr, new_size)`	Ridimensiona il blocco pre-esistente preservandone il contenuto.
`free(ptr)`	Rilascia il blocco nello Heap. Invocata due volte causa un **Undefined Behavior**.

5.3 Modificare Puntatori nelle Funzioni (Doppi Puntatori)

Se una funzione deve modificare il valore di un puntatore allocando memoria per conto del chiamante, occorre passare l'indirizzo del puntatore stesso tramite un **doppio puntatore (`Type**`)**:

Listing 2: Allocazione di Memoria tramite Doppio Puntatore

```c
void alloca_vettore(int n, int **ptr_v) {
    // Dereferenziando una volta (*ptr_v), aggiorniamo il puntatore originale del main
    *ptr_v = (int *) malloc(n * sizeof(int));
}

int main(void) {
    int *vec = NULL;
    alloca_vettore(10, &vec); // Passiamo l'indirizzo del puntatore
    if (vec != NULL) {
        vec[0] = 42;
        free(vec);
    }
    return 0;
}
```

Obiettivi: Comprendere a fondo la natura di un puntatore come variabile che memorizza un indirizzo di memoria, utilizzando correttamente gli operatori fondamentali di indirizzamento (`&`) e dereferenziazione (`*`). Acquisire padronanza nell'aritmetica degli indirizzi, e comprendere l'analogia tra array e puntatore, array-decay e passaggio di un array a funzione. Saper utilizzare correttamente le funzioni di libreria standard (`malloc, calloc, realloc, free`), interiorizzando **l'obbligo** del controllo di errore sul valore di ritorno (`NULL`) per evitare crash e segmentation fault. Comprendere la differenza tra puntatore `void *` (che è un tipo), e `NULL` (che è un valore), e dangling pointers e memory leak. Comprendere perché il meccanismo standard di passaggio per valore in C impedisce a una funzione di modificare direttamente un puntatore del chiamante.

6 Tipi Definiti dall'Utente e Strutture Collegate

6.1 Strutture (`struct`), Accesso ai Membri e Alignment

La `struct` è un tipo derivato che raggruppa variabili di tipo disomogeneo in un blocco contiguo di memoria. La parola chiave `typedef` crea un alias eliminando l'obbligo di ripetere `struct`.

- **Operatori di Accesso:** Si usa l'operatore punto (`.`) su variabili dirette; l'operatore freccia (`->`) su puntatori a strutture:

  $$\texttt{ptr} -> \texttt{campo} \iff (*\texttt{ptr}).\texttt{campo}$$

  Le parentesi in `(*ptr).campo` sono obbligatorie perché l'operatore di accesso `.` ha precedenza rispetto alla dereferenziazione `*`.

- **Struct Padding:** Per consentire alla CPU di accedere alla RAM allineandosi alle word dell'architettura (4/8 byte), il compilatore inserisce byte vuoti di riempimento (*padding*) tra i membri. Ordinando i campi all'interno della `struct` dal tipo con ampiezza maggiore al più piccolo si riduce al minimo lo spreco di memoria.

6.2 Unioni (`union`) ed Enumerazioni (`enum`)

- **Unioni (`union`):** Sintassi analoga alle strutture, ma i membri **condividono lo stesso indirizzo di memoria base**. La dimensione della `union` corrisponde a quella del suo membro più grande e contiene un solo valore valido alla volta.

- **Enumerazioni (`enum`):** Assegnano nomi simbolici parlanti a costanti intere (di default partono da 0 e incrementano di 1), migliorando la leggibilità nei blocchi `switch-case`.

6.3 Strutture Autoriferite e Liste Concatenate

Una struttura si definisce **autoriferita** quando contiene tra i propri membri un puntatore a un'altra struttura del medesimo tipo. Poiché la dimensione dell'oggetto diretto non sarebbe nota in fase di compilazione (dimensione infinita), è obbligatorio usare un puntatore (ampiezza fissa a 8 byte su architetture a 64-bit).

Listing 3: Creazione manuale di due nodi collegati tramite struttura autoriferita

```c
#include <stdio.h>
#include <stdlib.h>

// Struttura autoriferita per il nodo di una lista
typedef struct Node {
    int data;             // Campo informazione
    struct Node *next;    // Puntatore al nodo successivo
} Node;

int main(void) {
    // Allocazione del primo nodo (Testa) nell'Heap
    Node *head = (Node *) malloc(sizeof(Node));
    head->data = 10;

    // Allocazione del secondo nodo e collegamento manuale
    head->next = (Node *) malloc(sizeof(Node));
    head->next->data = 20;
    head->next->next = NULL; // Fine della catena

    // Deallocazione manuale della memoria (dall'ultimo al primo)
    free(head->next);
    free(head);

    return 0;
}
```

Ponte verso le Strutture Dati Avanzate: L'unione tra `struct` autoriferite, allocazione dinamica tramite `malloc` nell'Heap e la gestione dei doppi puntatori per le modifiche *in-place* costituisce la base fondamentale per realizzare strutture dati dinamiche avanzate come Liste Doppiamente Concatenate, Pile (LIFO), Code (FIFO), Alberi Binari e Grafi.

Obiettivi: Saper definire tipi strutturati complessi con `struct`, comprendere l'allineamento in memoria (*struct padding*) e realizzare strutture dati dinamiche elementari tramite strutture autoriferite e allocazione manuale dei nodi.